DECISION GOVERNANCE INFRASTRUCTURE

UNITED STATES PATENT AND TRADEMARK OFFICE

PROVISIONAL PATENT APPLICATION

TITLE OF INVENTION:

APPEND-ONLY POST-QUANTUM MERKLE TRANSPARENCY CHAIN FOR AUTOMATED GOVERNANCE DECISION RECEIPTS WITH INTERNAL RFC-3161-STYLE TIMESTAMPING, ROLLING HASH ACCUMULATION, AND TAMPER-EVIDENT PUBLIC VERIFICATION

Docket Number:	OMNIX-PAT-2026-010
Applicant / Inventor:	Harold Alberto Nunes Rodelo
Nationality:	United Kingdom
Entity Status:	Micro Entity (37 C.F.R. § 1.29)
Filing Basis:	35 U.S.C. § 111(b) — Provisional Application
Date Prepared:	April 19, 2026
Date of Filing:	April 19, 2026

This document constitutes a Provisional Patent Application filed pursuant to 35 U.S.C. § 111(b). It establishes a priority date and grants twelve (12) months from the filing date to submit a corresponding non-provisional application. This application has not been examined and does not confer patent rights independently. CONFIDENTIAL — NOT FOR DISTRIBUTION.

PROVISIONAL PATENT APPLICATION

OMNIX-PAT-2026-010

Title: APPEND-ONLY POST-QUANTUM MERKLE TRANSPARENCY CHAIN FOR AUTOMATED GOVERNANCE DECISION RECEIPTS WITH INTERNAL RFC-3161-STYLE TIMESTAMPING, ROLLING HASH ACCUMULATION, AND TAMPER-EVIDENT PUBLIC VERIFICATION

Inventor: Harold Alberto Nunes Rodelo

Applicant: OMNIX QUANTUM LTD

Filing Basis: 35 U.S.C. § 111(b)

Entity Status: Micro Entity

Date Prepared: April 19, 2026

Date of Filing: April 19, 2026

Related Applications: OMNIX-PAT-2026-001 (Governance Control Architecture), OMNIX-PAT-2026-003 (PQC Voice Auth)

FIELD OF THE INVENTION

The present invention relates to cryptographic audit trail systems for automated decision governance, and more particularly to an append-only transparency chain that links governance decision receipts using a rolling Merkle-style hash accumulator, applies post-quantum cryptographic signatures to each chain entry, generates RFC 3161-style trusted timestamps without dependency on external timestamp authorities, and exposes a public verification interface enabling external parties to verify the integrity and chronological ordering of the decision record without requiring access to the contents of individual receipts.

BACKGROUND

I. THE AUDIT TRAIL PROBLEM IN AUTOMATED GOVERNANCE

Automated decision systems operating in regulated environments must produce audit records that satisfy three independent regulatory requirements: completeness (every decision is recorded), integrity (records cannot be silently altered after the fact), and verifiability (third parties can confirm the integrity of the record without needing access to confidential decision contents).

Existing audit trail mechanisms fail one or more of these requirements:

1.1 Log-Based Systems: Traditional logging systems record decision events in timestamped log files. These systems fail the integrity requirement because log files can be modified after the fact without detection — there is no cryptographic linkage between log entries that would reveal retroactive modification.

1.2 Blockchain-Based Systems: Some systems use public or permissioned blockchains as immutable ledgers. These systems introduce significant operational dependencies: they require network connectivity to the blockchain, expose governance records to the blockchain's consensus and availability properties, and create regulatory uncertainty regarding the legal status of blockchain-recorded evidence in different jurisdictions.

1.3 External Timestamp Authorities (TSA): RFC 3161 timestamping requires submission of document hashes to external trusted timestamp authorities. This creates dependencies on third-party services, introduces network latency into the governance pipeline, and requires ongoing contractual and operational relationships with external parties.

1.4 Isolated Receipt Systems: Systems that generate individual cryptographic receipts per decision — without chaining those receipts together — provide per-receipt integrity but cannot detect the deletion of receipts from the record. An adversary who deletes a governance receipt leaves no detectable gap in the audit trail.

1.5 No Post-Quantum Security: All known prior art audit trail systems for automated governance use classical cryptographic primitives (RSA, ECDSA, Ed25519) for signatures and integrity verification. These primitives are vulnerable to quantum computing attacks capable of forging signatures and breaking hash function preimage resistance.

The present invention addresses all five inadequacies through the OMNIX Transparency Chain architecture.

SUMMARY OF THE INVENTION

The present invention provides an append-only transparency chain for automated governance decision receipts that:

- Chains governance receipts using a rolling Merkle-style hash accumulator, making deletion or modification of any entry detectable through chain verification.
- Applies post-quantum cryptographic signatures (in one embodiment, Dilithium-3 as specified in NIST FIPS 204) to each chain entry, ensuring that the chain remains tamper-evident against adversaries with quantum computing capabilities.
- Generates RFC 3161-style trusted timestamps internally without dependency on external timestamp authorities, eliminating the operational complexity and availability risk of third-party timestamping services.
- Exposes a public API endpoint enabling external verification of chain integrity without disclosing the contents of individual governance receipts — providing auditability while preserving confidentiality.
- Maintains backward compatibility with governance receipts generated before the transparency chain was deployed, allowing seamless integration into existing governance pipelines.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

I. SYSTEM ARCHITECTURE

The Transparency Chain (hereinafter "the System" or "TransparencyChain") is an evidence module that operates downstream of the governance checkpoint pipeline. Every governance receipt produced by the pipeline — whether representing a PASS, BLOCK, or conditional certification — is submitted to the Transparency Chain for recording.

Architectural position:

Governance Pipeline → Decision Receipt → Transparency Chain → Persistent Store + Public API

The System comprises four primary components: (1) the Internal Timestamp Module; (2) the Hash Chain Engine; (3) the Rolling Merkle Accumulator; and (4) the Public Verification Interface.

The System is applicable to any domain in which governance decisions must be recorded with tamper-evident integrity, including but not limited to: financial trading governance, clinical decision support audit trails, autonomous robotic action logs, energy grid dispatch records, insurance underwriting records, and any other domain subject to regulatory audit requirements.

II. INTERNAL TIMESTAMP MODULE — RFC 3161-STYLE WITHOUT EXTERNAL TSA

The Internal Timestamp Module generates trusted timestamps for governance receipts without requiring submission to an external Timestamp Authority (TSA). This is architecturally distinct from standard RFC 3161 timestamping in one critical way: the trusted timestamp is generated, signed, and verified entirely within the OMNIX governance infrastructure, with no dependency on external network services.

II.A. Timestamp Token Structure

For each governance receipt, the Internal Timestamp Module generates a Timestamp Token (TST) comprising:

- **version:** Protocol version identifier (in one embodiment: version 1)
- **hash_alg:** The hash algorithm used to compute the payload hash (in one embodiment: SHA-256)
- **payload_hash:** The SHA-256 hash of the governance receipt payload being timestamped
- **ts_utc:** A UTC ISO-8601 timestamp recording the moment of receipt submission
- **nonce:** A cryptographically random nonce (in one embodiment: 16 bytes, hex-encoded) preventing timestamp prediction and replay
- **policy:** An identifier for the timestamping policy under which the token was generated (in one embodiment: "OMNIX-ADR044-v1")

The TST is serialized to a canonical JSON representation with sorted keys, ensuring deterministic serialization across platforms and languages. A SHA-256 hash of the serialized TST (the TST hash) is computed and stored alongside the TST for verification purposes.

II.B. Timestamp Verification

To verify a Timestamp Token, the verifier re-serializes the TST data to its canonical JSON form, recomputes the SHA-256 hash, and compares it against the stored TST hash. Any modification to any field of the TST (including the timestamp, nonce, or payload hash) produces a different hash and is detected.

Rationale for internal timestamping: External TSAs introduce operational dependencies that are unacceptable in a governance pipeline that must operate with high availability. A governance system that cannot record receipts because an external TSA is unavailable fails its primary function. The internal timestamp mechanism provides equivalent integrity guarantees — the timestamp is cryptographically bound to the receipt payload and the chain entry — without any external dependency.

In other embodiments, the System may optionally submit timestamp tokens to an external TSA for additional regulatory assurance in jurisdictions where external TSA timestamps are legally required, without affecting the primary chain integrity mechanism.

III. HASH CHAIN ENGINE

The Hash Chain Engine creates a cryptographic linkage between successive chain entries, ensuring that no entry can be deleted, inserted, or modified without breaking the chain.

III.A. Chain Entry Structure

Each chain entry comprises:

- **log_id:** A unique identifier for the chain entry (in one embodiment: UUID v4)
- **receipt_id:** The identifier of the governance receipt being recorded
- **symbol / domain:** The operational domain or asset identifier for the decision
- **event_type:** Classification of the governance event (e.g., PASS, BLOCK, TENSIONED)
- **payload_hash:** SHA-256 hash of the governance receipt payload
- **prev_log_hash:** SHA-256 hash of the immediately preceding chain entry (or a canonical zero-hash for the first entry)
- **merkle_root:** The rolling Merkle root at the time this entry was appended
- **signing_provider:** Identifier of the cryptographic provider used to sign this entry
- **ts_utc:** UTC timestamp of chain entry creation
- **chain_version:** Protocol version for forward compatibility

III.B. Chain Linkage

The hash of each chain entry is computed over all fields including the prev_log_hash. This creates a one-directional cryptographic chain: each entry depends on all preceding entries. Modifying any entry in the chain changes its hash, which changes the prev_log_hash reference in the next entry, which changes that entry's hash, and so on. The modification propagates forward through the entire chain and is detectable at verification time.

Deletion detection: If an entry is deleted from the chain, the next entry's prev_log_hash no longer corresponds to any entry in the chain, producing an immediate verification failure.

Insertion detection: Inserting a new entry between two existing entries requires forging a post-quantum signature over the new entry — which is computationally infeasible with valid keys — and changing the prev_log_hash of the following entry, breaking its signature.

IV. ROLLING MERKLE ACCUMULATOR

The Rolling Merkle Accumulator maintains a compact running hash that accumulates the contribution of every receipt hash that has been appended to the chain. Unlike a static Merkle tree (which is computed over a fixed set of leaves), the rolling accumulator updates incrementally with each new entry, requiring no recomputation of the entire structure.

IV.A. Accumulation Formula

In one embodiment, the rolling Merkle root is computed as:

new_root = SHA256(prev_root || new_entry_hash)

where prev_root is the Merkle root of the chain before the new entry, new_entry_hash is the SHA-256 hash of the new chain entry, and || denotes concatenation.

For the first entry, prev_root is initialized to a canonical zero value (in one embodiment: a string of 64 zero characters representing a 256-bit zero hash).

In other embodiments, the accumulation function may be any collision-resistant combination function applied to the previous accumulator state and the new entry hash, provided that the function is deterministic and that any modification to any historical entry produces a different accumulated root value.

IV.B. Properties of the Rolling Accumulator

Incremental computation: The rolling Merkle root can be updated with O(1) computation per new entry, regardless of the total number of entries in the chain. This is architecturally critical for high-throughput governance pipelines where hundreds of decisions may be made per second.

Commitment to history: The current rolling Merkle root is a cryptographic commitment to the entire history of chain entries. Any modification to any historical entry changes the root value. The current root can be published (e.g., in a public API response, in a regulatory report, or in a timestamped attestation) as a compact proof of the current state of the complete chain.

Verification: A verifier who knows the initial root and the sequence of entry hashes can independently recompute the current root and compare it against the published root value.

V. POST-QUANTUM SIGNATURES ON CHAIN ENTRIES

Each chain entry is signed using the post-quantum cryptographic signing provider configured for the deployment. In one embodiment, entries are signed using Dilithium-3 (ML-DSA-65) as specified in NIST FIPS 204, providing security against adversaries with quantum computing capabilities.

The signing provider identifier is embedded in each chain entry, enabling the System to dispatch to the correct verification algorithm when verifying historical entries that may have been signed under a different provider than the current configuration (see Section VII on backward compatibility and crypto-agility).

The signed payload includes all chain entry fields, ensuring that the signature is invalidated by any modification to any field — including the prev_log_hash, the Merkle root, and the timestamp.

VI. PUBLIC VERIFICATION INTERFACE

The Public Verification Interface exposes an API endpoint (in one embodiment: `GET /api/transparency/chain`) that enables external parties — including regulators, auditors, clients, and independent researchers — to verify the integrity of the transparency chain without requiring access to the contents of individual governance receipts.

VI.A. Verification Without Content Disclosure

The public interface returns chain metadata including entry identifiers, timestamps, payload hashes, prev_log_hashes, Merkle roots, and signatures — but not the plaintext content of the governance receipts themselves. This enables:

- **Integrity verification:** A verifier can confirm that no entries have been deleted, inserted, or modified by recomputing the hash chain and Merkle root from the published hashes.
- **Completeness verification:** A client who possesses a governance receipt can verify that their receipt appears in the public chain by computing the receipt's payload hash and checking for a matching entry.
- **Chronological ordering verification:** The chain structure provides cryptographic evidence of the temporal ordering of governance decisions.

VI.B. Selective Disclosure

In one embodiment, the System supports selective disclosure: a governance receipt holder may present the plaintext of their receipt alongside the chain entry, enabling a verifier to confirm that the receipt's payload hash matches the chain entry's recorded hash, proving that the receipt was recorded in the chain at the claimed time.

VII. BACKWARD COMPATIBILITY

The System is designed to integrate into governance pipelines without requiring modification of existing receipt generation or verification mechanisms. Governance receipts generated before the Transparency Chain was deployed continue to be verifiable by existing receipt verification mechanisms.

The chain begins from the first receipt submitted after chain deployment. Receipts generated before this point do not appear in the chain and are not affected by chain verification. The chain version field in each entry documents the protocol version, enabling forward-compatible evolution of the chain format.

VIII. ALTERNATIVE EMBODIMENTS

8.1 Batched Merkle Trees: In one embodiment, entries are accumulated into a complete binary Merkle tree over fixed time windows (e.g., every 1,000 entries or every hour), with the Merkle root published and optionally submitted to an external timestamp authority for additional regulatory assurance.

8.2 Distributed Chain Replication: In one embodiment, the chain is replicated across multiple independent nodes, with consensus required before any new entry is accepted. The rolling Merkle root serves as a convergence point for replica synchronization.

8.3 Domain-Specific Sub-Chains: In one embodiment, independent sub-chains are maintained per operational domain (e.g., one chain for trading decisions, one for credit decisions, one for clinical decisions), with a master chain accumulating the roots of all sub-chains.

8.4 Configurable Hash Algorithms: In one embodiment, the hash algorithm used for chain construction is configurable (e.g., SHA-256, SHA-3-256, BLAKE3), with the algorithm identifier embedded in each entry for future-proof verification.

8.5 Cross-Domain Application: The Transparency Chain is equally applicable to any domain in which decisions must be audit-logged with tamper-evident integrity: clinical trial outcome records, autonomous vehicle action logs, robotic manufacturing decision trails, energy grid dispatch histories, and insurance claim processing records.

CLAIMS

Claim 1 (Broad — Core Chain) — A computer-implemented append-only transparency chain for automated governance decision records, comprising: a hash chain engine that links successive decision receipt records by incorporating the hash of each record into the computation of a chain linkage value for the subsequent record; a post-quantum signature module that applies a post-quantum cryptographic signature to each chain entry; and a verification module that detects modification, deletion, or insertion of any entry by verifying the continuity of hash linkage values across the chain.

Claim 2 (Specific — Rolling Merkle) — The system of Claim 1, further comprising a rolling Merkle accumulator that maintains a running commitment to the entire history of chain entries by combining the previous accumulator state with the hash of each new entry using a collision-resistant combination function, wherein the current accumulator value constitutes a compact cryptographic commitment to all prior entries.

Claim 3 (Specific — Rolling Formula) — The system of Claim 2, wherein, in one embodiment, the rolling accumulator is computed as new_root = SHA256(prev_root || new_entry_hash), and wherein in other embodiments any collision-resistant combination function may be substituted provided that modification of any historical entry produces a different accumulated root value.

Claim 4 (Broad — Internal Timestamp) — The system of Claim 1, further comprising an internal timestamp module that generates trusted timestamps for governance receipts without submission to an external timestamp authority, said timestamps comprising a cryptographically random nonce, a UTC timestamp, a hash of the receipt payload, and a policy identifier, wherein the timestamp token is integrity-protected by a hash of its canonical serialization.

Claim 5 (Broad — Public Verification) — The system of Claim 1, further comprising a public verification interface that enables external parties to verify chain integrity and confirm the presence of individual receipts in the chain without disclosing the plaintext content of individual governance receipts.

Claim 6 (Specific — PQC Signing) — The system of Claim 1, wherein, in one embodiment, each chain entry is signed using Dilithium-3 (ML-DSA-65) as specified in NIST FIPS 204, and wherein in other embodiments any post-quantum digital signature scheme may be substituted, with the signing provider identifier embedded in each entry to enable correct verification algorithm dispatch.

Claim 7 (Broad — Deletion Detection) — The system of Claim 1, wherein the hash linkage structure detects deletion of any entry by producing a hash linkage discontinuity at the position of the deleted entry, and wherein deletion of any entry is detectable without requiring comparison against an external reference.

Claim 8 (Broad — Backward Compatibility) — The system of Claim 1, wherein governance receipts generated before the chain was deployed remain verifiable through existing receipt verification mechanisms and are not required to appear in the chain, and wherein the chain begins from the first receipt submitted after chain deployment.

Claim 9 (Broad — Method) — A computer-implemented method for tamper-evident recording of automated governance decisions, comprising: receiving a governance decision receipt; computing a hash of said receipt; incorporating the previous chain linkage value into a new chain entry; computing a new rolling Merkle root combining the previous root with the new entry hash; signing the chain entry with a post-quantum cryptographic signature; and persisting the signed entry to an append-only store.

Claim 10 (Broad — System) — A computer-implemented system configured to provide tamper-evident audit evidence for automated governance decisions by: maintaining an append-only chain of post-quantum-signed governance receipt records linked by cryptographic hash values; providing a rolling Merkle accumulator value as a compact commitment to the complete decision history; generating trusted timestamps without external timestamp authority dependency; and exposing a public verification interface enabling integrity verification without content disclosure.

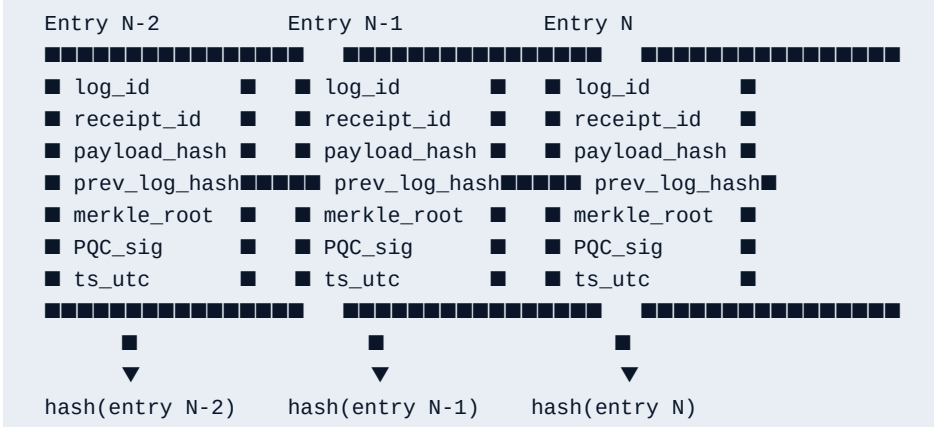
ABSTRACT

An append-only transparency chain provides tamper-evident audit records for automated governance decisions across any institutional domain. Each governance decision receipt is recorded as a chain entry containing the receipt's payload hash, the hash of the preceding entry, a rolling Merkle accumulator value, a post-quantum cryptographic signature (in one embodiment: Dilithium-3, NIST FIPS 204), and an internally generated RFC 3161-style trusted timestamp requiring no external timestamp authority. The rolling Merkle accumulator — computed in one embodiment as SHA256(prev_root || new_entry_hash) — provides a compact commitment to the entire decision history that updates in O(1) time per entry. Modification, deletion, or insertion of any chain entry is detected through hash linkage verification without reference to an external record. A public API exposes chain metadata for external integrity verification and selective disclosure by receipt holders without revealing receipt contents. The system maintains backward compatibility with receipts generated before

chain deployment. Applications include financial trading governance, clinical decision support, autonomous robotics, energy grid management, and any regulated domain requiring tamper-evident decision audit trails. To the inventor's knowledge, this is the first system combining rolling Merkle accumulation, internal RFC-3161-style timestamping without external TSA dependency, and post-quantum signatures specifically for automated governance decision audit chains.

DRAWINGS

FIG. 1 — Transparency Chain Structure



Modify any field → hash changes → next entry's prev_log_hash invalid
Delete any entry → hash linkage gap → chain verification fails

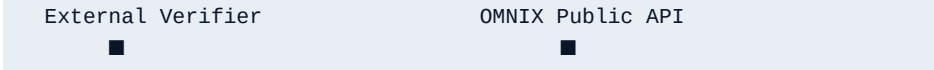
FIG. 2 — Rolling Merkle Accumulator

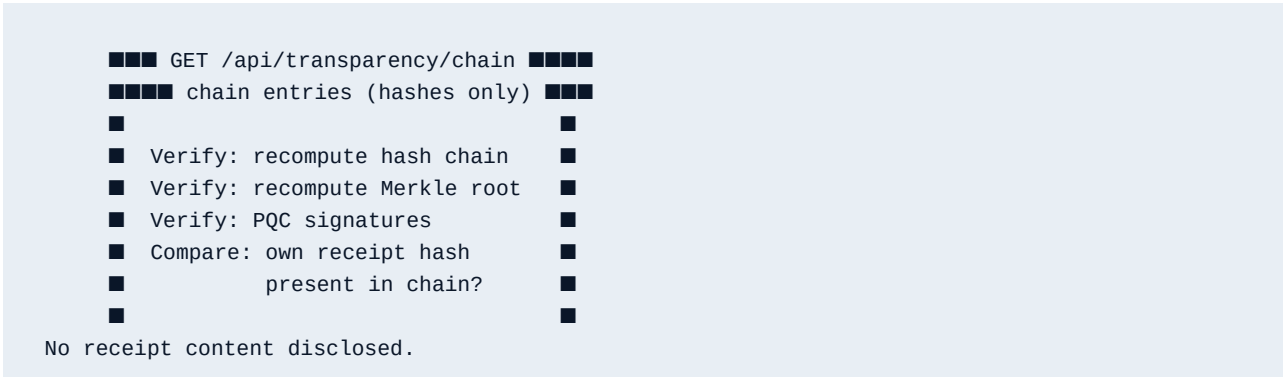
```
root_0 = "0"×64
root_1 = SHA256(root_0 || hash(entry_1))
root_2 = SHA256(root_1 || hash(entry_2))
root_N = SHA256(root_{N-1} || hash(entry_N))

Current root_N = commitment to ALL N entries
O(1) update per new entry
Published root enables third-party verification
```

FIG. 3 — Internal Timestamp Token

```
TST = {
  version:      1,
  hash_alg:     "SHA-256",
  payload_hash: SHA256(receipt_payload),
  ts_utc:       "2026-04-19T...",
  nonce:        <16 random bytes>,
  policy:       "OMNIX-ADR044-v1"
}
tst_hash = SHA256(canonical_json(TST))
No external TSA required.
```

FIG. 4 — Public Verification Flow




GET /api/transparency/chain
chain entries (hashes only)
Verify: recompute hash chain
Verify: recompute Merkle root
Verify: PQC signatures
Compare: own receipt hash
present in chain?
No receipt content disclosed.